Kiến trúc VNOJ

Các thành phần, nhiệm vụ, lý do tồn tại và phương án thay thế

Phân tích từ source code `VNOI-Admin/OJ` (site) và `VNOI-Admin/judge-server`

11/11/2026

1 Phạm vi và nguồn đã dùng

Tài liệu này chỉ nói về **kiến trúc**: hệ thống gồm những khối nào, mỗi khối làm gì, vì sao cần, thiếu thì sao, và có thể thay hoặc bỏ không.

Tôi đã kiểm tra độ mới của từng repo trước khi dùng:

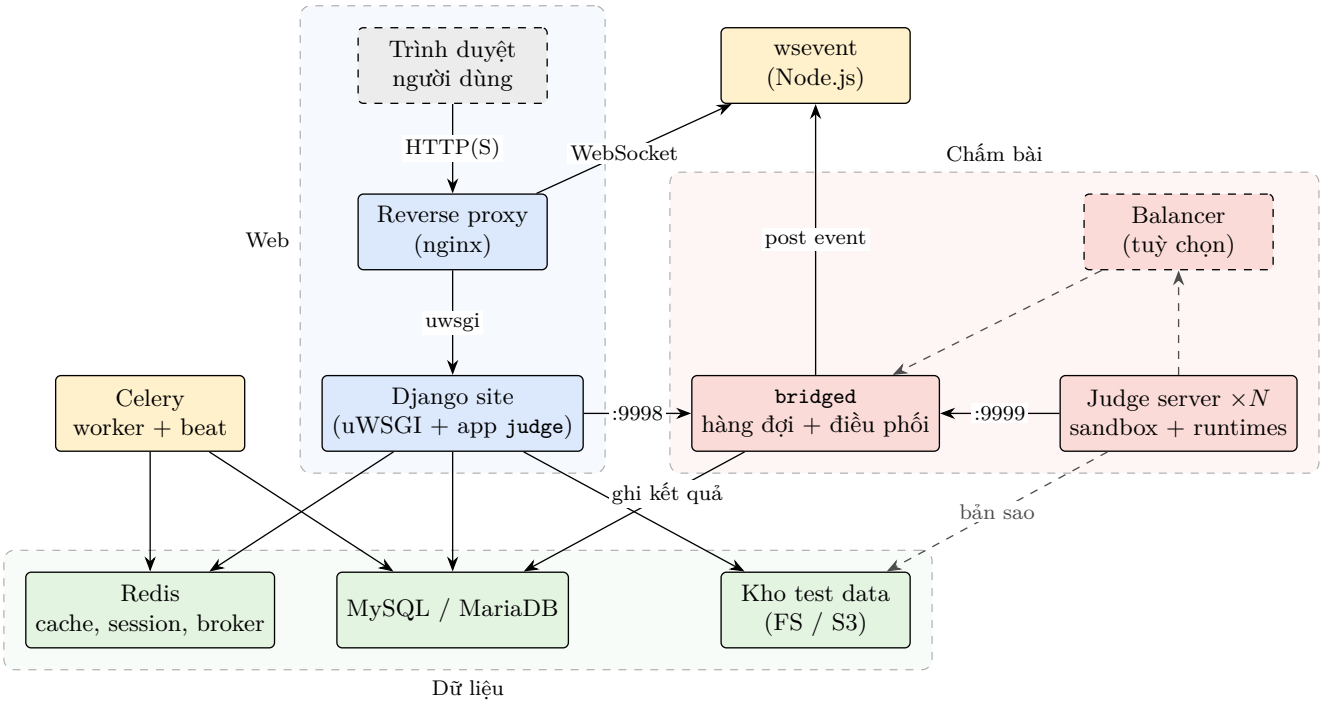
Repo	Commit mới nhất	Dùng?	Ghi chú
`OJ` (site Django)	04/11/2026	**Có, căn cứ chính**	Đã có S3 storage, judge protocol v2, celery beat, OAuth.
`judge-server` (`master`)	10/10/2026	**Có, căn cứ chính**	Cùng bản đã cài `tier-vnoj` ở phần trước.
`vnoj-docker`	13/09/2023	Chỉ để tham khảo cách ghép nginx/uWSGI	**Cũ**: không có S3, không chạy celery beat. Mọi chỗ trích từ đây đều ghi rõ.
`vnoj-docs`	19/09/2024	Không	Cũ hơn site khoảng 2 năm và không mô tả kiến trúc.

Các nhánh chưa merge của `judge-server` (`microvm-api-judge`, `hiep/landlock`, `wip/parallel-case-grading`) **không** được tính là kiến trúc hiện hành; chúng chỉ được nhắc ở mục sandbox.

Giới hạn: trang `oj.vnoi.info` nằm sau Cloudflare challenge (HTTP 403, `cf-mitigated: challenge`), nên tôi **không quan sát được hạ tầng chạy thật** (số máy chấm, vị trí đặt từng khối, CDN). Mọi kết luận dưới đây là về thiết kế trong mã nguồn, không phải về cách VNOI đang vận hành.

2 Bức tranh tổng thể

VNOJ là bản fork của DMOJ. Có thể chia thành bốn mặt phẳng: *web* (phục vụ người dùng), *dữ liệu*, *bất đồng bộ / thời gian thực*, và *chấm bài*. Điểm quan trọng nhất: **Django không tự chấm bài**. Nó giao việc cho `bridged`, và `bridged` giao cho các máy chấm (judge) kết nối vào.



Hình 1. Kiến trúc tổng thể. Nét đứt = tuỳ chọn hoặc suy ra từ code. Cổng `:9998`: Django gửi `submission-request` cho `bridged`. Cổng `:9999`: judge là bên chủ động mở kết nối tới `bridged`, nên máy chấm không cần mở cổng ra ngoài. "Bản sao": judge đọc test data từ storage đã khai báo (cách đồng bộ chưa xác minh).

3 Từng thành phần

Mỗi khối trả lời năm câu hỏi: làm gì, vì sao cần, thiếu thì sao, thay hoặc bỏ được không, và căn cứ ở đâu trong source.

A. Mặt phẳng web

A1. Reverse proxy (nginx)

Làm gì	Nhận HTTP(S) từ người dùng. Phục vụ file tĩnh `/static`. Chuyển request động sang uWSGI (`uwsgi_pass`). Proxy WebSocket `/event/` và long-poll `/channels/` sang `wsevent`. Phục vụ file tải lớn do Django uỷ quyền (`X-Accel-Redirect`: `userdatacache`, `contestdatacache`).
Vì sao cần	App server không nên phục vụ file tĩnh và giữ hàng nghìn kết nối WebSocket dài. Proxy cũng cho một địa chỉ duy nhất cho web và realtime.
Nếu không có	Site vẫn chạy được ở chế độ dev, nhưng mất: phục vụ static hiệu quả, đường vào chung cho WebSocket, tải file nội bộ. Production không nên bỏ.
Thay thế / bỏ?	**Thay được**: Caddy, Traefik, Apache, load balancer của cloud, hoặc đặt sau CDN. Nếu bỏ uWSGI thì phải đổi `uwsgi_pass` thành `proxy_pass`. Repo `OJ` không bắt buộc nginx; nó chỉ là cách ghép trong ví dụ triển khai.
Căn cứ (source)	`vnoj-docker/dmoj/nginx/conf.d/nginx.conf` (**2023, cũ**, chỉ dùng để minh hoạ). Cấu hình `DMOJ_USER_DATA_INTERNAL` trong `dmoj/settings.py` xác nhận cơ chế `X-Accel-Redirect` vẫn còn.

A2. Django site (uWSGI + app `judge`)

Làm gì	Toàn bộ logic nghiệp vụ: tài khoản, bài tập, kỳ thi (các format: `default`, `ioi`, `legacy_ioi`, `icpc`, `atcoder`, `ecoo`, `vnoj`), nộp bài, bảng xếp hạng, rating, ticket, blog, tổ chức, trang admin, API v2, OAuth (Django OAuth Toolkit). Khi có bài nộp, nó lưu vào DB rồi gửi yêu cầu chấm cho `bridged`.
Vì sao cần	Đây là sản phẩm VNOJ. Mọi dữ liệu và luật nghiệp vụ nằm ở đây.
Nếu không có	Không còn hệ thống.
Thay thế / bỏ?	**Không bỏ được.** Chỉ lớp phục vụ WSGI là thay được (uWSGI ↔ gunicorn). Có hai chế độ vận hành đặc biệt trong cấu hình: `VNOJ_LOW_POWER_MODE` (giới hạn truy vấn nặng khi tải cao) và `VNOJ_OFFICIAL_CONTEST_MODE` (tắt một số tính năng khi thi chính thức).
Căn cứ (source)	`judge/`, `dmoj/settings.py`, `dmoj/wsgi.py`, `judge/contest_format/`.

B. Mặt phẳng dữ liệu

B1. MySQL / MariaDB

Làm gì	Nguồn dữ liệu duy nhất (source of truth): user, problem, submission, từng test case, tham gia kỳ thi, bảng `Judge` (tên, `auth_key`, tier, online), ngôn ngữ. **Cả Django lẫn** `bridged` **đều ghi vào đây**; `bridged` cũng đọc bảng `Judge` để xác thực máy chấm.
Vì sao cần	Cần lưu trạng thái bền vững, truy vấn quan hệ và giao dịch. Hai tiến trình (site, bridged) dùng chung trạng thái qua DB.
Nếu không có	Không chạy được.
Thay thế / bỏ?	**Không bỏ được.** Thay thế thực tế: MariaDB hay MySQL. File cấu hình mẫu ghi "only MySQL is supported" và các `contest_format` chứa SQL thô, nên đổi sang DB khác cần kiểm tra kỹ. SQLite chỉ là mặc định cho dev/CI.
Căn cứ (source)	`dmoj/settings.py` (`DATABASES`), `judge/models/`, `judge/bridge/judge_handler.py` (`_authenticate`).

B2. Redis

Làm gì	Ba việc: (1) cache của Django (`django_redis`); (2) lưu session (`SESSION_ENGINE = cached_db`); (3) broker và result backend của Celery.
Vì sao cần	Giảm tải DB (ví dụ cache thanh điều hướng, danh sách bài). Celery cần một hàng đợi trung gian giữa web và worker.
Nếu không có	Không có cache dùng chung thì mỗi tiến trình uWSGI có cache riêng, dẫn đến dữ liệu không nhất quán. Không có broker thì Celery không chạy.
Thay thế / bỏ?	**Thay được** nhưng phải đổi cả ba chỗ: cache sang memcached (README của OJ nhắc memcached hoặc Redis); broker sang RabbitMQ (repo có `pika` và `event_poster_amqp.py`); session có thể dùng DB thuần.
Căn cứ (source)	`dmoj/settings.py` (`CACHES`, `SESSION_ENGINE`), `CELERY_BROKER_URL` trong cấu hình triển khai.

B3. Kho dữ liệu đề (Problem data storage)

Làm gì	Lưu test data của bài (file `init.yml`, `.in`/`.out`, archive). Có hai backend: `filesystem` và `s3`, chọn qua `DMOJ_STORAGE_CONFIG_PATH` (YAML) hoặc `DMOJ_PROBLEM_DATA_ROOT`. Site lưu metadata để khỏi đọc archive ở mỗi lần xem. Khi gửi yêu cầu chấm, site kèm trường `storage` (`problem.effective_storage`); judge chỉ nhận bài khi đã khai báo storage đó lúc handshake.
Vì sao cần	Judge phải có test data để so sánh output; site cần nơi để người ra đề tải lên.
Nếu không có	Không có test data thì không chấm được.
Thay thế / bỏ?	**Thay được**: filesystem cục bộ, thư mục dùng chung (NFS), hoặc S3. Có thêm dịch vụ lưu trữ ngoài tuỳ chọn (`VNOJ_PROBLEM_ARCHIVE_SERVICE_URL`). **Chưa xác minh**: cơ chế đồng bộ dữ liệu từ site sang máy chấm trong production (bản sao cục bộ trên judge hay đọc chung).
Căn cứ (source)	`judge/utils/problem_data_storage.py`; commit "support S3 storage for problem data" (26/09/2026); `judge/bridge/judge_handler.py` (`can_judge`).

C. Bất đồng bộ và thời gian thực

C1. Celery worker + beat

Làm gì	Chạy các việc nặng hoặc định kỳ ngoài request web. Tác vụ: rejudge hàng loạt (`rejudge_problem_filter`), tính lại điểm (`rescore_problem`, `rescore_contest`), chạy MOSS (`run_moss`), xuất dữ liệu (`prepare_contest_data`, `prepare_user_data`), thông báo kỳ thi, webhook Discord. **Beat** có 4 lịch: `queue_time_stats` (hằng ngày), `problem_garbage_collect`, `update_hot_problems` (10 phút/lần), `organization_monthly_reset` (đầu tháng).
Vì sao cần	Các việc này chạy lâu; đặt trong request web sẽ làm treo trang. Việc định kỳ cần một bộ lập lịch.
Nếu không có	**Chấm bài vẫn chạy bình thường**, vì đường chấm đi thẳng Django → `bridged` bằng TCP, không qua Celery. Mất: rejudge hàng loạt, rescore, export, MOSS, webhook, và danh sách "hot problems" không được cập nhật (từ 20/10/2026 nó được tính bằng beat).
Thay thế / bỏ?	**Bỏ được ở mức site tối thiểu**, không nên bỏ ở production. Thay thế: cron + `manage.py`; hoặc chạy task đồng bộ (tính năng `task_always_eager` của Celery, chưa thử trên VNOJ). Lưu ý: ví dụ `vnoj-docker` cũ chỉ chạy `worker`, **không có beat**, nên cấu hình triển khai theo ví dụ đó sẽ không chạy các lịch trên.
Căn cứ (source)	`dmoj/celery.py` (`beat_schedule`), `judge/tasks/*.py`; `vnoj-docker/dmoj/celery/Dockerfile` (2023).

C2. wsevent (event server, Node.js)

Làm gì	Đẩy sự kiện đến trình duyệt theo thời gian thực. Có ba cổng: `15100` (trình duyệt nhận qua WebSocket), `15101` (Django và `bridged` gửi sự kiện vào), `15102` (long-poll dự phòng). Kênh dùng: `sub_<secret>` (một bài nộp), `submissions`, `contest_<id>`, ticket, thông báo. Giữ vài chục tin gần nhất trong RAM để client bắt kịp.
Vì sao cần	Người dùng thấy bài nộp chuyển *Queued* → *Grading* → *kết quả từng test* mà không phải tải lại trang; bảng xếp hạng kỳ thi tự cập nhật.
Nếu không có	Nếu đặt `EVENT_DAEMON_USE = False`, hàm `post()` thành no-op. Site và chấm bài vẫn đúng, chỉ mất cập nhật trực tiếp (phải F5).
Thay thế / bỏ?	**Bỏ được** mà không ảnh hưởng tính đúng. Thay thế có sẵn trong repo: đường AMQP (`event_poster_amqp.py`, dùng RabbitMQ). Có thể thay bằng polling phía trình duyệt, nhưng sẽ tăng tải lên web.
Căn cứ (source)	`judge/event_poster.py`, `websocket/daemon.js`, `dmoj/settings.py` (`EVENT_DAEMON_*`).

D. Mặt phẳng chấm bài

D1. `bridged` (cầu nối chấm bài)

Làm gì	Chạy bằng `manage.py runbridged`. Mở **hai server TCP**: cổng `9998` dành cho Django (nhận `submission-request`, `terminate-submission`, `disconnect-judge`, `disable-judge`) và cổng `9999` dành cho máy chấm. Việc chính: • **Xác thực** judge (handshake, so `auth_key` trong DB; chỉ nhận protocol version ≥ 2). • **Hàng đợi ưu tiên** 4 mức: thi (0) > thường (1) > rejudge (2) > rejudge hàng loạt (3); có khái niệm *tier* của judge và giữ lại một judge rảnh khỏi rejudge. • **Ghép** bài nộp với judge phù hợp (đúng storage, đúng ngôn ngữ, không bị cấm trong kỳ thi), gắn thời gian/bộ nhớ giới hạn lấy từ DB. • **Ping**, phát hiện judge im lặng (20 giây không ack thì ngắt). • **Nhận kết quả và ghi DB**: tạo `SubmissionTestCase`, gộp batch, tính điểm tổng, cập nhật điểm người dùng, thống kê bài, điểm kỳ thi, rồi gửi sự kiện cho `wsevent`.
Vì sao cần	Judge không nối thẳng DB và không nên tin cậy. Cần một cổng xác thực và điều phối trung tâm, và cần một nơi **tổng hợp verdict**. Chú ý: verdict và điểm cuối cùng do `bridged` tính, không phải judge.
Nếu không có	**Không có chấm bài**: bài nộp sẽ không bao giờ rời trạng thái *Queued*. Phần còn lại của site vẫn hoạt động.
Thay thế / bỏ?	**Không bỏ được.** Trong repo không có phương án thay thế; protocol của judge-server gắn với nó. Về nguyên tắc có thể thiết kế lại bằng hàng đợi thông điệp (RabbitMQ/Kafka) và judge kéo việc, nhưng phải viết lại cả hai phía. **Rủi ro thiết kế**: hàng đợi nằm trong RAM của một tiến trình (điểm lỗi đơn). Khi khởi động, nó đặt mọi judge `online=False` và chuyển các bài ở trạng thái `QU/P/G` sang `IE`.
Căn cứ (source)	`judge/bridge/daemon.py`, `django_handler.py`, `judge_handler.py`, `judge_list.py`, `judge/judge_priority.py`, `judge/judgeapi.py`.

D2. Judge server (`judge-server`, image `tier-vnoj`)

Làm gì	Một tiến trình chạy trên máy chấm. Nó **kết nối ra** `bridged`, gửi handshake (`id`, `key`, danh sách executor và phiên bản runtime, danh sách storage), rồi chờ `submission-request`. Với mỗi bài: biên dịch, chạy từng test **trong sandbox** (ptrace, `cptbox`), cho checker so output, rồi gửi lại các gói `grading-begin`, `compile-message` hoặc `compile-error`, `test-case-status` (từng test), `grading-end`, `internal-error`. Có thể chạy nhiều judge song song. Image `vnoj/judge-tiervnoj` = runtimes tier 1 (C/C++, Python, Java 8, Pascal) + OpenJDK 25, Go, Kotlin, Rust, PyPy2, `scratch-run`, `testlib`.
Vì sao cần	Chạy code của thí sinh là việc nguy hiểm và nặng CPU; tách ra máy riêng có sandbox giúp cô lập rủi ro và mở rộng ngang bằng cách thêm máy.
Nếu không có	Bài nộp ở lại *Queued* cho đến khi có một judge kết nối.
Thay thế / bỏ?	**Không bỏ được**, nhưng số lượng và cách đặt thì linh hoạt (0 đến N). Sandbox: bản `master` dùng ptrace; các nhánh `hiep/landlock` và `feat/microvm-api-judge` là hướng thay thế **chưa merge**. **Lưu ý cấu hình**: protocol v1 bị site từ chối; judge phải đặt `judge_version: 2` và khai báo `problem_storage_globs` theo `id/glob`. (Bản web test ở phần trước chạy chế độ v1 cục bộ, không nối với site.)
Căn cứ (source)	`dmoj/judge.py`, `dmoj/packet.py`, `dmoj/judgeenv.py`, `dmoj/cptbox/`, `.docker/tiervnoj/Dockerfile`.

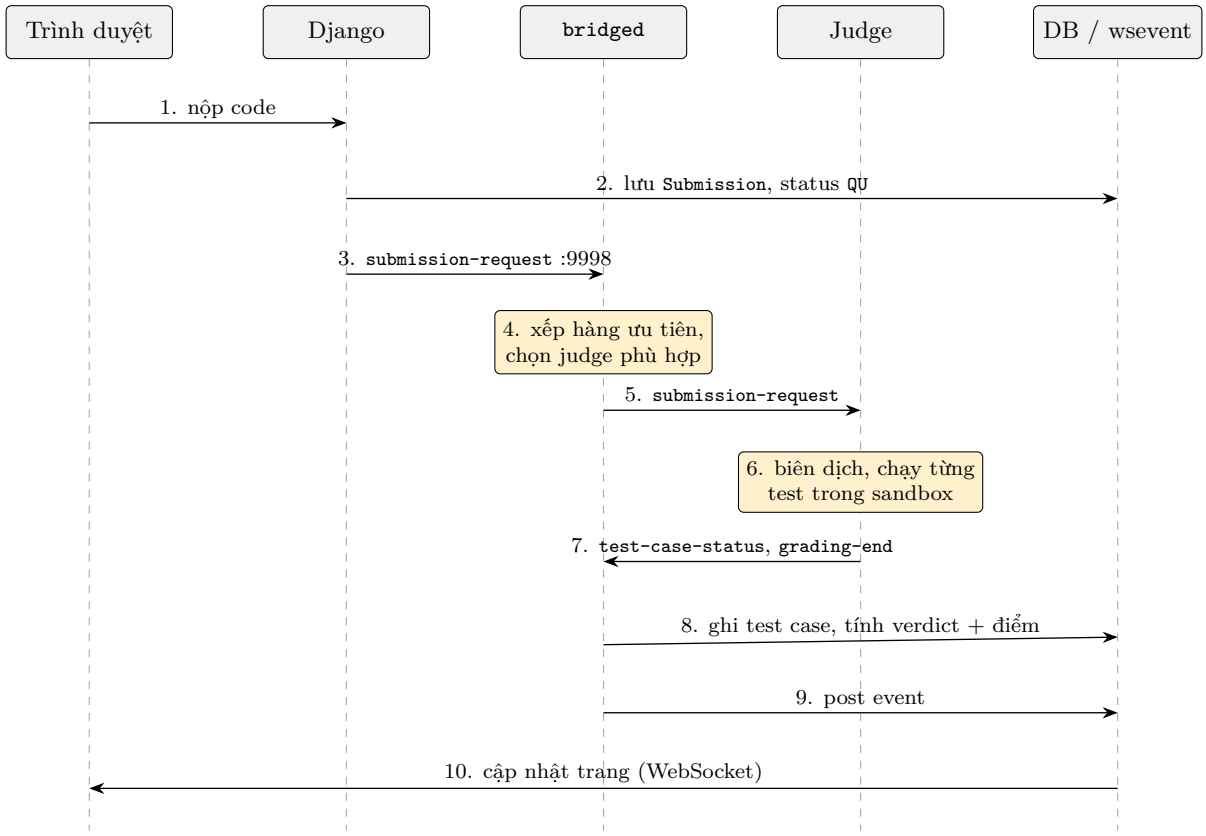
D3. Balancer (tuỳ chọn)

Làm gì	Chạy bằng `manage.py runbalancer`. Nó đứng giữa: judge kết nối vào balancer (cổng `8888`) như thể đó là `bridged`, còn balancer kết nối ra *nhiều* `bridged` khác nhau (mỗi bridge một cặp `id`/`key`) và ghép một judge với một bridge tại một thời điểm.
Vì sao cần	Cho phép một dàn máy chấm dùng chung phục vụ nhiều site hoặc nhiều kỳ thi mà không phải cấu hình lại từng judge. *(Suy ra từ code; chưa có tài liệu mô tả.)*
Nếu không có	Không ảnh hưởng nếu chỉ có một site: judge nối thẳng `bridged`.
Thay thế / bỏ?	**Bỏ được** với hệ thống một site.
Căn cứ (source)	`judge/balancer/*.py`, `judge/management/commands/runbalancer.py`.

E. Dịch vụ ngoài (đều tuỳ chọn)

Dịch vụ	Làm gì	Nếu không có
PDF (`pdfoid`, `DMOJ_PDF_PDFOID_URL`)	Dịch vụ ngoài render đề bài ra PDF.	Site không tự render PDF từ đề. (Đề vẫn có thể đính kèm file PDF có sẵn: commit 27/10/2026 cho phép xoá PDF statement trong form sửa bài.)
Archive service (`VNOJ_PROBLEM_ARCHIVE_SERVICE_URL`)	Dịch vụ ngoài xử lý lưu trữ gói đề.	Dùng storage thường.
MOSS (`MOSS_API_KEY`)	Kiểm tra đạo văn code.	Mất tính năng chống đạo văn.
Discord webhook	Thông báo ticket, bài mới, kỳ thi, hàng đợi dài.	Mất thông báo.
Email (SMTP)	Đăng ký, đặt lại mật khẩu.	Không gửi được email.
Pandoc (trong image site)	Dùng khi nhập gói Polygon (`judge/utils/codeforces_polygon.py`).	Không nhập được gói Polygon.

4 Luồng một bài nộp qua kiến trúc



Hình 2. Vòng đời một bài nộp. Bước 2 và 8 cùng ghi vào DB; bước 8 do `bridged` thực hiện chứ không phải Django hay judge.

5 Bảng quyết định: bỏ hay thay?

Thành phần	Bắt buộc?	Nếu bỏ thì mất gì	Thay bằng
Django site	**Có**	Mất toàn bộ hệ thống	Không
DB (MySQL/MariaDB)	**Có**	Không chạy được	MariaDB ↔ MySQL
`bridged`	**Có** (để chấm)	Bài nộp mãi ở *Queued*	Không có trong repo
Judge server	**Có** (để chấm)	Như trên	Không; sandbox khác còn ở nhánh chưa merge
Kho test data	**Có** (để chấm)	Không có dữ liệu để so	Filesystem, NFS, S3
Reverse proxy	Production có	Mất static, WS, tải nội bộ	Caddy, Traefik, Apache, LB cloud
Redis	Thực tế có	Cache và session lệch giữa tiến trình, không có broker	memcached + RabbitMQ
Celery (worker + beat)	Không	Mất rejudge hàng loạt, rescore, export, MOSS, hot problems, thông báo	cron + `manage.py`; chạy eager
wsevent	Không	Mất cập nhật trực tiếp, đúng/sai vẫn đúng	AMQP (RabbitMQ), polling
Balancer	Không	Chỉ ảnh hưởng khi dùng chung judge cho nhiều site	Judge nối thẳng `bridged`
PDF, MOSS, Discord, Email	Không	Mất từng tính năng riêng	Không thay hoặc bỏ

6 Điểm cần lưu ý về kiến trúc

1. `bridged` **là điểm lỗi đơn.** Hàng đợi nằm trong RAM; khởi động lại sẽ đặt mọi bài đang `QU/P/G` thành `IE` (cần rejudge).
2. **Verdict do** `bridged` **tổng hợp**, nên lỗi tính điểm hoặc batch nằm ở site, không ở judge-server.
3. **Phải chạy Celery beat** nếu muốn các lịch trong `dmoj/celery.py` hoạt động; cấu hình theo ví dụ `vnoj-docker` cũ không có beat.
4. **Khoá mặc định trong** `settings.py` (`EVENT_DAEMON_*_KEY`, `GLOBAL_API_KEY = 'test-api-key-123'`) là giá trị mẫu; production phải ghi đè trong `local_settings.py`.
5. **Phiên bản protocol**: site từ chối judge v1. Judge-server mặc định `judge_version` là 1, nên phải đặt tường minh bằng 2.
6. **Phần chưa xác minh**: cách đồng bộ test data tới từng máy chấm, số lượng và vị trí máy chấm của VNOI, và việc VNOI có chạy balancer hay không.